Comparative Performance Evaluation of TF-IDF, BM25, Sentence Transformer and Hybrid RRF Retrieval for Legal Holdings on CaseHOLD

Students Name	Registration No.	Email-id
Vaibhav Bansal	23FE10CSE00333	VAIBHAV.23FE10CSE00333@muj.manipal.edu
Pranav Joshi	23FE10CSE00701	Pranav.23FE10CSE00701@muj.manipal.edu
Ishaan Manocha	23FE10CSE00143	

Abstract: Lawyers and judges routinely cite earlier cases for a specific holding, and finding that holding quickly is a retrieval problem. We turn the CaseHOLD benchmark into a retrieval task in which the passage that cites a case is the query and the pool of 13,297 holding statements in the test split is the document collection. We compare TF-IDF, Okapi BM25, a Sentence Transformer (all-MiniLM-L6-v2) and a hybrid that merges their rankings with Reciprocal Rank Fusion (RRF), with every setting chosen on the validation split. On 3,600 test queries BM25 reaches an MRR of 0.323 and the off-the-shelf dense model only 0.234. The three-way RRF hybrid improves Recall@100 from 0.710 to 0.761 but gives only a small MRR gain (0.328). Fine-tuning the dense model for one epoch on the CaseHOLD training split raises its MRR to 0.389, and fusing it with BM25 gives our best result (MRR 0.400, Recall@10 0.597). A general-purpose cross-encoder re-ranker made results worse, which shows how much domain matters in legal text.

Keywords: *Information Retrieval · CaseHOLD · BM25 · Sentence Transformers · Reciprocal Rank Fusion · Legal Search*

1. Introduction

Legal arguments are built on precedent. When a court writes that other courts “have held” something, it backs the sentence with a citation, and in US case law that citation is usually followed by a short parenthetical that states what the cited case decided, its holding. A lawyer drafting a brief has the opposite problem: they know what point they want to make and need to find the case whose holding supports it. Doing this by hand through thousands of decisions is slow, so it is a natural job for an information retrieval (IR) system.

Classical IR handles this with sparse lexical matching. TF-IDF and Okapi BM25 score a document by the query words it contains, weighted by how rare those words are, and an inverted index makes the lookup fast. Their known weakness is vocabulary mismatch. A citing passage may talk about “fears of a gas pipeline” while the holding talks about “stigma damages”. Dense retrievers built on Sentence Transformers map both texts to vectors and compare meaning instead of words, but a model trained on general web text may not know legal language, and it can miss exact terms such as statute names that BM25 catches easily. Reciprocal Rank Fusion (RRF) is a simple way to combine both kinds of ranker using only their rank positions.

Our project (Project 13 in the course list) builds and evaluates such a system on CaseHOLD [4], a dataset of more than 53,000 citing passages from US court decisions, each paired with the correct holding and four wrong ones. CaseHOLD is normally used as a multiple-choice reading task. We recast it as retrieval: the citing passage is the query and every holding in the split is a document. We then compare the four ranking models required by the brief (TF-IDF, BM25, Sentence Transformer and hybrid RRF) using Precision@10, Recall, MAP and nDCG, and draw Precision-Recall graphs.

Beyond the required models we tried two extensions that came up while reading the results: fine-tuning the Sentence Transformer on the CaseHOLD training split, and re-ranking the top results with a cross-encoder. We also built a small search interface so that any passage can be typed in and the four rankers compared side by side. All parameters were chosen on the validation split and every number in this report comes from one run of our code on the test split.

2. About Dataset

2.1 Visualization of the Dataset and an Example

CaseHOLD (Case Holdings On Legal Decisions) was released by Zheng et al. [4]. It was built from the Harvard Law Library case law corpus. For each citation followed by a parenthetical that starts with "holding", the authors kept the text before the citation as the prompt, replaced the holding with a <HOLDING> marker, and stored the real holding as the right answer. The four wrong answers are other holdings from the corpus that have the highest TF-IDF similarity to the right one, chosen on purpose to make the task hard. A law student scored 94% on the task in the original paper.

We load the version packaged in the LexGLUE benchmark [5] (Hugging Face: coastalcph/lex_glue, config case_hold), which has fixed train, validation and test splits. Table 1 gives the sizes and Figure 1 the length distributions. The queries are long (about 136 words) and the holdings short (about 25 words), so a query shares only a handful of terms with its answer. In the test split the 18,000 answer options contain 13,297 distinct texts, and 1,032 correct holdings also appear as a wrong option for some other query, so a holding can be right for one passage and a distractor for another. The correct answer is spread evenly over the five positions (between 713 and 727 test questions per position), so no position bias can be used.

Table 1. CaseHOLD (LexGLUE version) split sizes as used in this project.

Split	Citing contexts (queries)	Unique holdings (documents)	Avg. words per query	Avg. words per holding
Train	45,000	50,291	136.3	26.4
Validation	3,900	14,138	136.3	25.4
Test	3,600	13,297	136.4	25.4

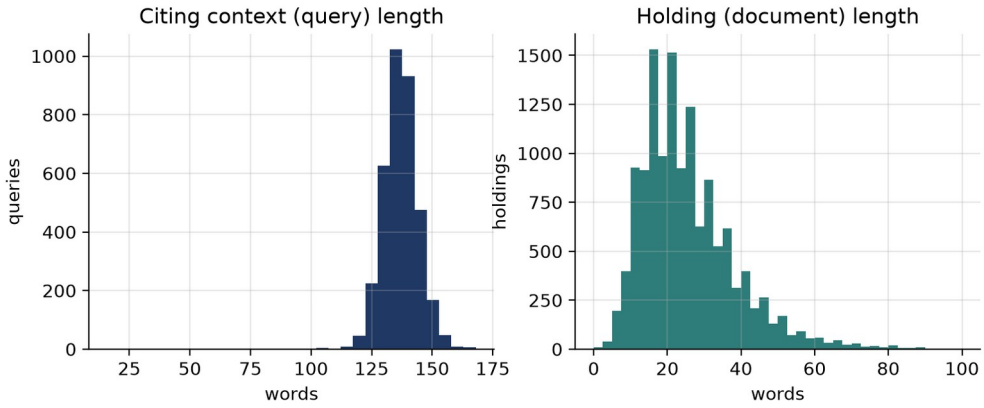


Figure 1: Length of the citing contexts (queries) and of the unique holdings (documents) in the test split.

The box below shows test query 0, shortened at the start. The correct answer is choice 4. Choices 2 and 3 are on the same topic (severance and stigma damages), which is exactly what makes the multiple-choice version hard for word-matching methods.

> **Citing context:** … Other courts, in the same or similar circumstances, have held that the anticipation of increased interference or potential hazards on the land remaining after a taking, which one court has labeled “stigma damages,” can be considered in assessing severance damages. See, e.g., Vector Pipeline, L.P., v. 68.55 Acres of Land, 157 F.Supp.2d 949, 957 (N.D.Ill.2001) (<HOLDING>); …
>
> **0:** holding that actual damages under respa included emotional damages
>
> **1:** holding that increased fear of flooding was a matter that factfinder could properly consider in assessing severance damages
>
> **2:** holding that local buyers fear of possible knapweed infestation legitimately affected calculation of severance damages
>
> **3:** recognizing concept of stigma damages as they apply to property which is in fact contaminated
>
> **4 (correct): holding that stigma damages were properly included in calculation of severance damages where proximity of gas pipeline could trigger fears about possible mishaps**

2.2 Turning CaseHOLD into a Retrieval Task

We evaluate every model in two settings. In the **full-pool setting** the documents are all 13,297 unique holdings of the test split and each of the 3,600 queries has exactly one relevant document. This is the realistic search setting. In the **5-choice setting** each query is scored only against its own five options, which matches the original CaseHOLD task. Here we report accuracy (the correct holding ranked first), so random guessing gives 0.20.

3. Methodology

Figure 2 shows the pipeline. The query and the holdings go through two paths. The sparse path cleans and stems the text, builds an inverted index and scores with TF-IDF and BM25. The dense path sends the raw text to a Sentence Transformer and compares vectors with cosine similarity. The ranked lists are merged with RRF. An optional cross-encoder step re-orders the top 20 of the fused list.

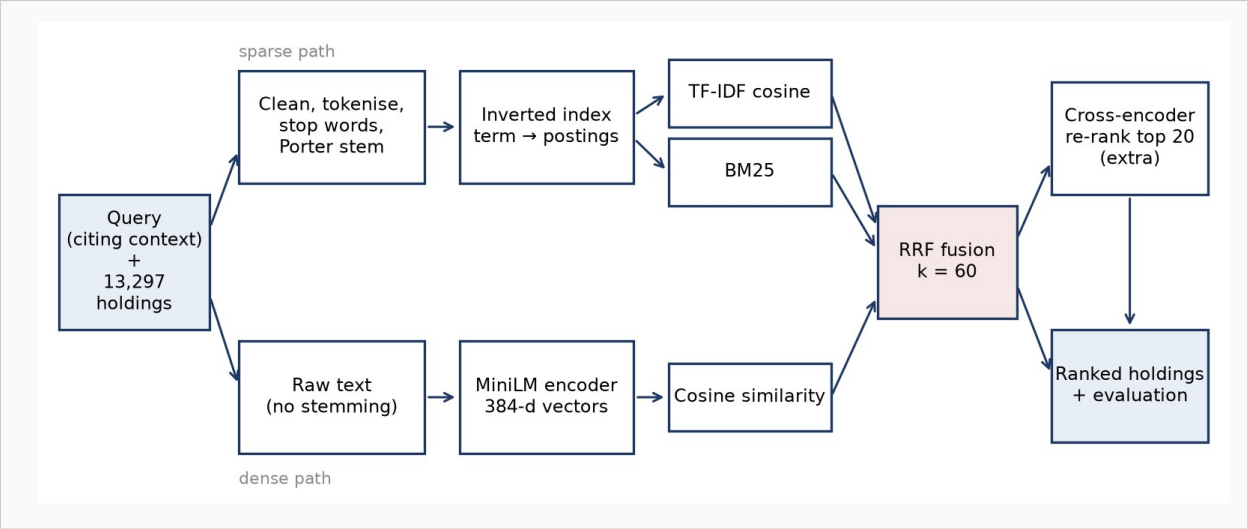


Figure 2: Retrieval pipeline used in this project, with a sparse path (TF-IDF, BM25) and a dense path (Sentence Transformer) merged by Reciprocal Rank Fusion.

3.1 Preprocessing and Indexing

For the sparse models every query and holding is lower-cased, the <HOLDING> marker is removed and dotted abbreviations are joined into one token ("e.g." becomes "eg"). The text is split on anything that is not a letter or digit, and we drop one-letter tokens and every token that contains a digit. The second rule removes reporter citations such as 157 F.Supp.2d 949, which are unique to each case and only add noise. We then remove the scikit-learn English stop-word list plus a few legal words that occur everywhere (holding, held, v, see, eg, id, cf, supra, citing; 91% of holdings start with "holding that") and apply the Porter stemmer [9]. For the example above, the holding becomes: stigma, damag, properli, includ, calcul, sever, damag, ...

The inverted index maps each stemmed term to a postings list of (holding id, term frequency) pairs and stores each holding's length. On the test pool it has 7,233 terms and 150,284 postings; an average holding has 12.56 indexed terms and an average postings list has 20.78 entries. The dense model does not use this index. It reads the raw text, because stemming and stop-word removal would destroy the sentences it was trained on.

3.2 Ranking Models

We evaluate the four models required by the brief and two extensions:

- **TF-IDF Vector Space Model:** each text becomes a vector with weight (1 + log tf) x idf per term, normalised to unit length; holdings are ranked by cosine similarity with the query (scikit-learn TfidfVectorizer with our tokenizer).
- **Okapi BM25:** written by us on top of the inverted index. Term frequency is saturated by k1 and normalised by document length through b. We checked that the matrix version used for batch evaluation gives the same scores as a plain loop over the postings lists (differences below 1e-5, which is float rounding).
- **Sentence Transformer (Dense Search):** all-MiniLM-L6-v2, a 6-layer model that outputs 384-dimensional vectors [2, 6]. Queries and holdings are encoded once and compared by cosine similarity. Inputs longer than 256 word pieces are cut off by the model.
- **Hybrid Reciprocal Rank Fusion (RRF):** merges the top-1000 lists of TF-IDF, BM25 and the dense model (Section 3.3). We also report BM25 + dense without TF-IDF.
- **Extension 1, fine-tuned dense model:** all-MiniLM-L6-v2 trained for one epoch on the 45,000 training pairs (citing context, correct holding) with one wrong option as a hard negative, using the multiple negatives ranking loss [7] (batch 32, learning rate 2e-5, 11 minutes on the laptop GPU).
- **Extension 2, cross-encoder re-ranker:** cross-encoder/ms-marco-MiniLM-L6-v2 [8] reads each (query, holding) pair together and re-scores the top 20 of the hybrid list.

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{idf}(t) \cdot \mathrm{tf}(t,d) \cdot (k1 + 1) \,/\, \left(\mathrm{tf}(t,d) + k1 \cdot (1 - b + b \cdot |d| \,/\, \mathrm{avgdl}) \right)$$

where idf(t) = ln(1 + (N − df(t) + 0.5) / (df(t) + 0.5)), N is the number of holdings, df(t) the number of holdings containing t, |d| the length of the holding and avgdl the average length.

3.3 Reciprocal Rank Fusion Formulation

RRF [3] adds up the reciprocal of the position each ranker gives a document. Given a set of rankers M and a constant k, the RRF score of document d is:

$$\mathrm{RRF_Score}(d \in D) = \sum_{m \in M} 1 \,/\, (k + r_m(d))$$

where r_m(d) is the 1-based rank of d in the list of model m, and a document missing from a list adds nothing. RRF needs no score normalisation, which matters here because BM25 scores are unbounded (around 40 for the example query) while cosine scores lie between −1 and 1. We use the standard k = 60 for the main results and report the k chosen on validation as an extra run.

3.4 Choosing Parameters on the Validation Split

All choices were made on the 3,900 validation queries and then frozen. Using the whole context as the query worked best for BM25 (MRR 0.307) compared with only the last 64 or 32 words (0.290 and 0.214). A grid over k1 and b (Figure 3) picked k1 = 0.6 and b = 0.9; a low k1 and a high b suit short documents where repeating a term carries little extra meaning. For TF-IDF, stop-word removal alone did not help (0.279 against 0.283) but adding stemming did (0.306). Between the two small dense models, all-MiniLM-L6-v2 (0.234) beat bge-small-en-v1.5 (0.182 at best, even with its query instruction). For RRF, smaller k was slightly better (k = 1: 0.331, k = 60: 0.317).

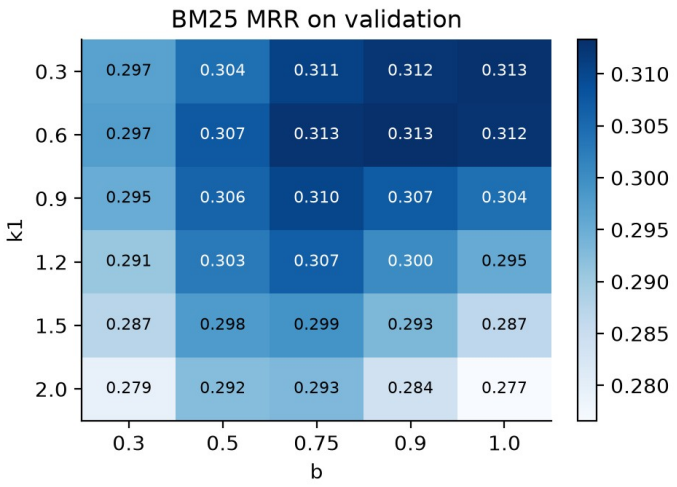


Figure 3: BM25 MRR on the validation split for different k1 and b.

3.5 Evaluation Metrics

Each query has one relevant holding, so all metrics depend on the rank r of that holding. Precision@10 is 1/10 if r ≤ 10 and 0 otherwise, so its maximum possible value here is 0.1, not 1. Recall@k is the share of queries with r ≤ k. Average precision of a query equals 1/r, so MAP is the same as Mean Reciprocal Rank (MRR). nDCG@10 with binary relevance is 1/log2(r + 1) when r ≤ 10. The Precision-Recall graph plots mean Precision@k against mean Recall@k for k = 1 to 100. We also tested every model against BM25 with a Wilcoxon signed-rank test on per-query reciprocal ranks.

4. Experimental Setup and Performance Analysis

Everything ran on one laptop (Apple M4 Pro, 24 GB RAM) with Python 3.13, scikit-learn, sentence-transformers and PyTorch; the neural models used the Apple GPU (MPS). Building the TF-IDF and BM25 indexes took 0.26 s and 0.24 s, and scoring all 3,600 test queries took about 0.6 s for each. Encoding the queries and holdings with the dense model took 14 s and cross-encoder re-ranking took 198 s. Table 2 gives the full-pool results with the metrics of the brief, Table 3 the 5-choice results.

Table 2. Full-pool retrieval on the CaseHOLD test split (3,600 queries, 13,297 holdings). Maximum possible Precision@10 is 0.1.

Ranking Model	Precision@10	Recall@100	MAP (= MRR)	nDCG@10
TF-IDF	0.049	0.729	0.317	0.349
BM25	0.049	0.710	0.323	0.356
Sentence Transformer	0.037	0.605	0.234	0.260
Hybrid RRF (TF-IDF + BM25 + Dense)	0.052	0.761	0.328	0.365
Hybrid RRF, k = 1	0.053	0.759	0.342	0.379
Hybrid RRF (BM25 + Dense)	0.049	0.747	0.314	0.348
Hybrid + cross-encoder	0.041	0.761	0.196	0.231
Dense, fine-tuned (ext.)	0.058	0.797	0.389	0.426
Hybrid RRF (BM25 + fine-tuned Dense)	**0.060**	**0.817**	**0.400**	**0.440**

BM25 is the strongest of the required single models (MAP 0.323), with TF-IDF just behind (0.317). The general-purpose Sentence Transformer is clearly worse (0.234): it was trained on web text and question answering pairs, not on court language, and it puts the correct holding at median rank 33 compared with 11 for BM25. Even so, the hybrid helps. The three-way RRF raises Recall@100 from 0.710 to 0.761 and Recall@10 from 0.493 to 0.521, while MAP rises only a little (0.328). The fusion mostly pulls answers from deep in the list into the top 100, which is what matters if the list is handed to a person or a re-ranker. With the k picked on validation the gain is larger (MAP 0.342). Dropping TF-IDF from the fusion lowers every metric, because with only two inputs the weaker dense list gets as much weight as BM25.

The two extensions behave very differently. One epoch of fine-tuning on the training split lifts the dense model from 0.234 to 0.389, ahead of BM25, and fusing it with BM25 gives the best run (MAP 0.400, Recall@100 0.817). The ms-marco cross-encoder, on the other hand, drops MAP to 0.196. It was trained to judge whether a web passage answers a short search question, and a 140-word legal passage paired with a holding is far from that. According to the Wilcoxon test, the improvements of the three-way hybrid ($p = 5.7e\text{-}17$), the fine-tuned model and the fine-tuned hybrid over BM25 are statistically significant, while BM25 + Dense is not different from BM25 ($p = 0.46$).

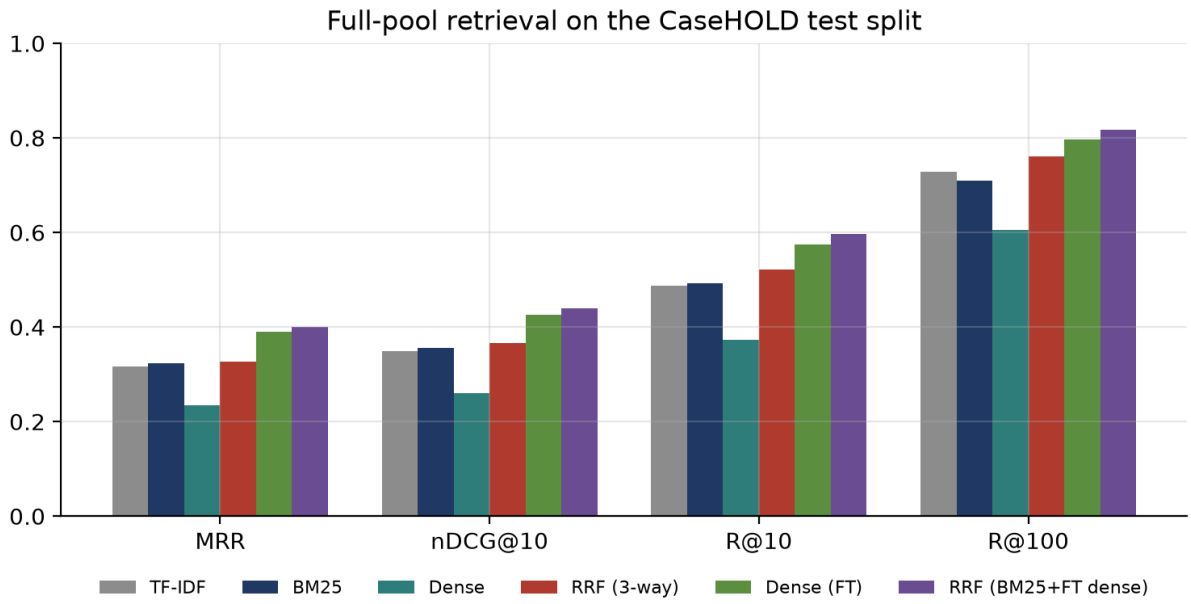


Figure 4: Full-pool results for the main runs.

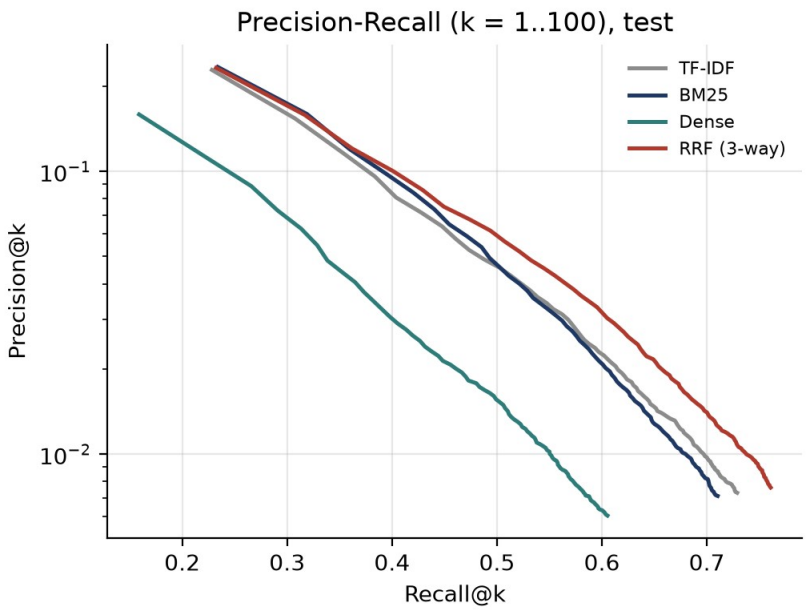


Figure 5: Precision-Recall graph for the four required models (each point is mean Precision@k against mean Recall@k, k = 1 to 100; log scale on precision).

Figure 5 is the Precision-Recall graph asked for in the brief. Because there is one relevant holding per query, precision falls roughly as 1/k as the cut-off grows, so we plot it on a log scale. The hybrid curve is level with BM25 for the first two cut-offs and above it from k = 3 (recall about 0.36), the same effect seen in Recall@100. The dense curve is lower everywhere. Figure 6 shows where the correct holding ends up for each run.

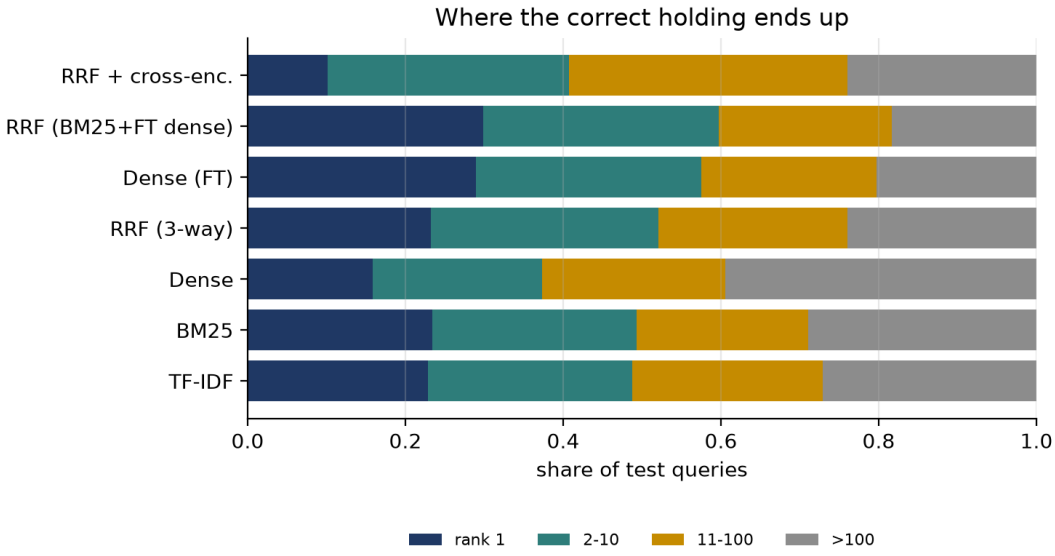


Figure 6: Share of test queries whose correct holding is ranked first, 2 to 10, 11 to 100, or below 100.

Table 3. 5-choice setting (original CaseHOLD format) on the test split. Random guessing gives accuracy 0.20.

Ranking Model	Accuracy	MRR	Recall@2	nDCG@5
TF-IDF	0.565	0.735	0.777	0.802
BM25	0.590	0.755	0.807	0.817
Sentence Transformer	0.526	0.707	0.741	0.780
Hybrid RRF (TF-IDF + BM25 + Dense)	0.583	0.745	0.782	0.809
Hybrid RRF (BM25 + Dense)	0.576	0.742	0.787	0.807
Cross-encoder (ms-marco)	0.417	0.632	0.658	0.724
Hybrid RRF (BM25 + fine-tuned Dense)	0.610	0.768	0.825	0.827
Dense, fine-tuned (ext.)	**0.634**	**0.783**	**0.838**	**0.838**

In the 5-choice setting every method is far above chance, but the ordering changes. BM25 (0.590) is now better than the three-way hybrid (0.583), and the fine-tuned dense model alone is best (0.634). The four wrong options were picked by TF-IDF similarity to the right answer, so all five candidates share vocabulary and the lexical signal that RRF adds is less useful. For context, Zheng et al. [4] report a macro F1 of about 0.4 for a BiLSTM and about 0.6 for fine-tuned BERT on their own split. Our numbers are accuracies on the LexGLUE split, so they are not directly comparable, but they place unsupervised BM25 and our small fine-tuned model in a reasonable range.

4.1 Error Analysis

The sparse and dense models fail on different queries. BM25 ranks the correct holding first while the dense model does not in 474 queries; the reverse happens in 204; both succeed in 368. This is why fusion helps at all. In 1,571 queries (44%) neither puts it in the top 10. When the hybrid misses rank 1, the holding it puts first is one of the query's own four distractors in only 13% of cases, and its word overlap with the correct holding is low (Jaccard 0.09). Longer holdings are easier: BM25 MRR rises from 0.213 for holdings of 15 words or fewer to 0.402 for holdings above 40 words, since there are more terms to match.

Three test queries show the typical cases. In query 3185 the passage is about money **structuring** offences; BM25 ranks the right holding first because of that rare word, while the dense model ranks it 129th and returns a holding about habitual felony sentencing. In query 296 the passage talks about

relationships and a ban on contact; BM25 ranks the answer 1,694th, but the dense model ranks it first because it links the context to “governmental actions restricting police officers sexual conduct” without any shared keyword. Query 2885 fails everywhere: the passage is about a strip search but the cited holding is about one panel of a court being bound by an earlier panel. The citation was used for a procedural point that the surrounding text never mentions, and no ranker that only reads the context can recover that.

4.2 Search Interface

For the demonstration we built a Streamlit app (app/search_app.py). The user picks a CaseHOLD test query or types any passage, and the app shows the preprocessed query terms and the top results of TF-IDF, BM25, the dense model (base or fine-tuned) and the RRF hybrid next to each other, marking the correct holding when it is known. Single queries are answered by walking the BM25 postings lists directly and take well under a second.

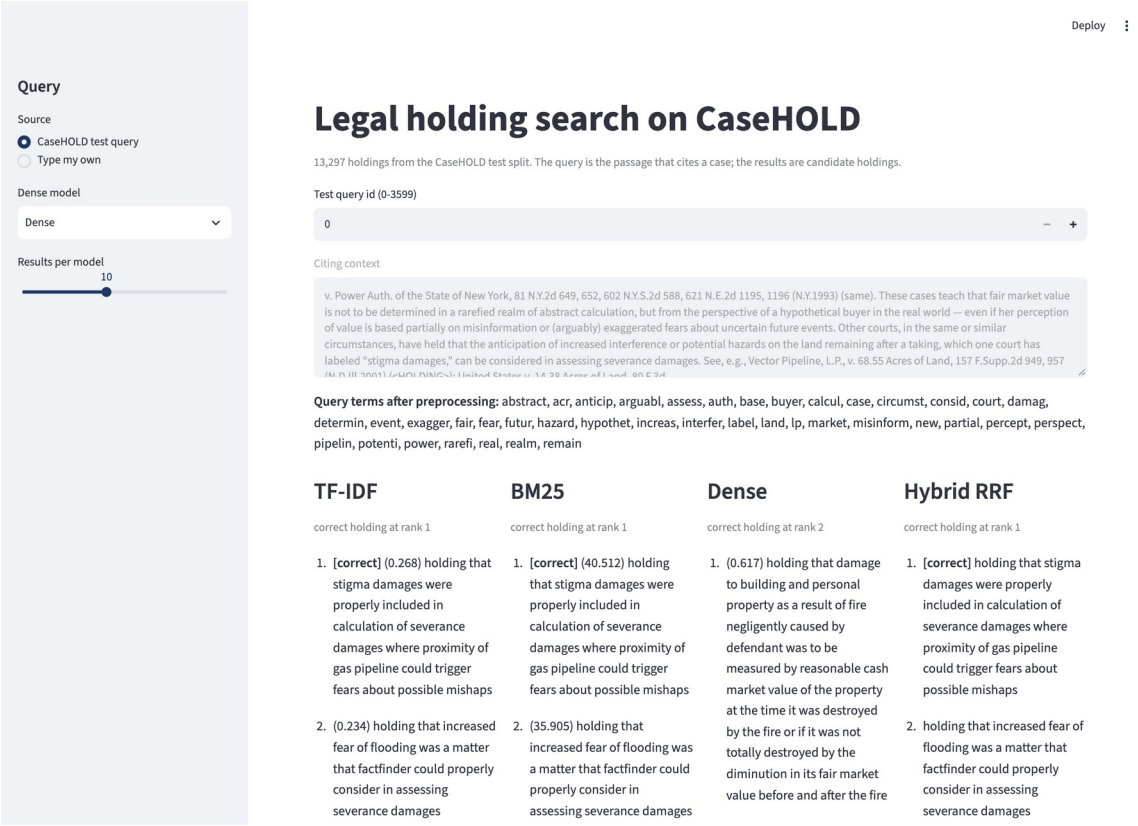


Figure 7: Search interface for test query 0. TF-IDF, BM25 and the hybrid rank the correct holding first; the dense model ranks it second.

5. Conclusion

We turned CaseHOLD into a legal holding retrieval task and compared TF-IDF, BM25, a Sentence Transformer and an RRF hybrid on it. The main lessons are:

- **Lexical models are strong on legal text.** BM25 with tuned k1 and b (MAP 0.323) beat an off-the-shelf dense model by a wide margin (0.234).
- **Fusion helps recall more than the top rank.** Three-way RRF raised Recall@100 to 0.761 but MAP only slightly, because the dense list is weak on its own.
- **Domain data matters most.** Fine-tuning the small dense model on CaseHOLD training pairs moved its MAP from 0.234 to 0.389, and BM25 plus the fine-tuned model was best overall (0.400). A cross-encoder trained on web search hurt.

The work has limits. Each query has exactly one relevant holding, so Precision@10 is capped at 0.1 and a holding that is legally correct but not the cited one counts as wrong. The document pool is the 13,297 holdings of the test split, not a full case law database, and the wrong options inside it were chosen to be similar, so it is harder than a random pool of the same size. We used small models and one epoch of training because everything had to run on a laptop. In future work we would try a legal-domain encoder such as Legal-BERT [4], train the cross-encoder on CaseHOLD itself, and weight the rankers inside RRF instead of treating them equally.

References

1. Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 333–389 (2009)
2. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)
3. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In: Proceedings of SIGIR, pp. 758–759 (2009)
4. Zheng, L., Guha, N., Anderson, B.R., Henderson, P., Ho, D.E.: When Does Pretraining Help? Assessing Self-Supervised Learning for Law and the CaseHOLD Dataset of 53,000+ Legal Holdings. In: Proceedings of ICAIL (2021)
5. Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D.M., Aletras, N.: LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In: Proceedings of ACL (2022)
6. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In: Proceedings of NeurIPS (2020)
7. Henderson, M., et al.: Efficient Natural Language Response Suggestion for Smart Reply. arXiv:1705.00652 (2017)
8. Nogueira, R., Cho, K.: Passage Re-ranking with BERT. arXiv:1901.04085 (2019)
9. Porter, M.F.: An algorithm for suffix stripping. Program 14(3), 130–137 (1980)
10. Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press (2008)